GPU-Accelerated Parallel BFS: Linear-Algebraic vs. Graph-First on a Single A100

Aakaash Nattanmai | Harshaan Chugh

CS 5220: Applied High-Performance and Parallel Computing • Cornell University • NERSC Perlmutter (NVIDIA A100)

1. Motivation

Breadth-First Search (BFS) is the workhorse of graph analytics – shortest paths, connected components, centrality, web crawling, social-network and brain-connectome pipelines all depend on a fast BFS primitive.

Yet BFS is notoriously *hard to parallelize* on GPUs:

- Irregular, data-dependent memory access patterns.
- Severe load imbalance on skewed-degree graphs.
- Frontier sizes that vary by orders of magnitude per level.

The community has converged on **two** competing GPU paradigms. This poster compares both on identical hardware, inputs, and CSR storage so the algorithmic differences are isolated from implementation noise.

2. Two Paradigms

(a) Linear-Algebraic (SpMV-based). Express each BFS level as a sparse matrix–vector multiply on the adjacency matrix A:

$$f_{k+1} = (A \cdot f_k) \odot \neg v_k,$$

where f_k is the level-k frontier (dense or bitmap) and v_k the visited mask. Decades of sparse-LA optimization apply directly.

(b) Graph-First (Direction-Optimizing). Operate on graph topology with explicit *push* (top-down) and *pull* (bottom-up) kernels, switching adaptively based on frontier size [1]. Closer to traditional CPU BFS, avoids reformulating BFS as linear algebra.

3. Research Questions

1. Which paradigm wins on which graph topology (chain, sparse-random, dense-random, power-law)?
2. Are the classical optimizations (*warp-cooperation, bitmap frontier, push-pull*) equally beneficial when expressed as SpMV vs. as graph traversal?
3. What is the runtime cost (or savings) of the *linear-algebra abstraction* compared to a hand-tuned graph kernel?

4. Example: Warp-Cooperative Pull

From `spmv_bfs_warp`: one *warp* (32 threads) scans one vertex's neighbors in parallel, votes with `__ballot_sync`, and a single thread writes if any neighbor was on the frontier.

```
// One warp = 32 threads working on ONE vertex i.
// row_start/row_end = neighbor indices in col_idx; lane = thread id 0..31.
for (int e = row_start + lane; e < row_end; e += 32) {
    if (frontier[col_idx[e]]) {   // neighbor on frontier?
        hit = 1;
        break;
    }
}
// Did ANY thread see a hit? ballot_sync collects one bit per thread.
unsigned mask = __ballot_sync(0xFFFFFFFF, hit);
// Lane 0 writes once if any lane found a frontier neighbor.
if (mask && lane == 0) {
    new_frontier[i] = 1;
    visited[i] = 1;
    depth[i] = level;
    atomicOr(found_new, 1);
}
```

5. Experimental Setup

Storage. Compressed Sparse Row (CSR): `row_ptr[V+1]`, `col_idx[2E]`. Edges are loaded once on the host, undirected (each edge inserted in both directions), then transferred to GPU global memory.

Hardware. Single NVIDIA A100 (40GB) on NERSC *Perlmutter*. Compute capability `sm_80`; compiler `nvcc -O2`. Restricted to `CUDA_VISIBLE_DEVICES=0` for fair single-GPU comparison.

Validation. Every variant is checked level-by-level against a reference Python BFS (`validate_bfs.py`). Only kernel time is measured (CUDA events); graph load and I/O are excluded.

Graph	$\lvert V\rvert$	$\lvert E\rvert$	$\bar{d}$
`m_chain`	1K	999	2
`m_sparse`	1K	3K	6
`m_dense`	1K	50K	100
`l_sparse`	100K	500K	10
`l_dense`	100K	5M	100
`l_power`	100K	1M	40

Synthetic graphs from `generate_graphs.py` (RMAT for power-law). Exact $\lvert E\rvert$ in repo scripts.

6. Linear-Algebraic SpMV BFS

Think of each BFS round as updating "who is on the frontier next" using the adjacency matrix. We ship four kernel versions you can select at run time:

Baseline. One thread per vertex. It walks that vertex's neighbors; if any neighbor is on the frontier, it turns on for the next round. Easy to understand, but one very high-degree vertex can hold up its whole group of 32 threads.

Warp-cooperative rows. Give **one vertex** to a **team of 32 threads** (a warp). They **split** its neighbor list: thread 0 checks neighbors $\#0, 32, 64, \ldots$; thread 1 checks $\#1, 33, 65\ldots$; and so on. So a "hub" with thousands of edges shares work across the team instead of one thread doing the whole list while the other 31 wait.

Bitmap frontier. Store "on frontier" as single bits instead of full integers so we read and write less data.

Push–pull. *Push:* only frontier vertices touch neighbors. *Pull:* every unvisited vertex checks whether any neighbor is on the frontier. Each round we estimate an **edge budget** for the frontier—roughly **frontier size** × **average degree**—and compare it to **all edges in the graph**. If that budget is **still below about one-fourteenth** of total edges, we **push**; if it crosses above that, we **pull**. So the switch is "how edge-heavy is this frontier compared to the whole graph?"—not a fixed vertex count.

7. Graph-First Direction-Opt. BFS

Beamer's direction-optimizing BFS [1]: a per-level decision between two kernels.

Top-down (push). Each thread owns one frontier vertex and pushes itself onto every unvisited neighbor. `atomicCAS` on `visited[]` prevents duplicate enqueues. Optimal when the frontier is small (early/late BFS levels).

Bottom-up (pull). Each thread owns one unvisited vertex and stops as soon as it finds *any* neighbor in the frontier. Optimal in the middle of BFS when most edges already point to the frontier.

Switching heuristic. With m_f the sum of out-degrees of frontier vertices, switch top-down → bottom-up when $m_f > |E|/\alpha$ (we use α=14), and back to top-down when $|f_k| < |V|/\beta$ (β=24).

Implementation. The GPU driver keeps each frontier as both a compact queue and a bitmap. Both push and pull kernels produce the next queue, next bitmap, next frontier size, and next frontier edge count entirely on device; the host copies only those two counters for the next direction decision.

Enhancement to try. *Two-level staging:* replicate very small frontiers into `__shared__` bitmap slices and run short vectorized loads for the pull kernel, so hub neighborhoods that repeat across rounds hit L1/shared before global memory (not implemented).

8. Results (head-to-head summary)

Median of N=10 runs on one Perlmutter A100 (CUDA events; load/print excluded). On every synthetic in block **5**, the fastest SpMV kernel beats graph-first in our current timings; gaps are tight on sparse/dense random graphs (about 1.1–1.3×) and widen on power-law and chain stress cases.

Per-kernel SpMV sweep (all variants in ms) is tabulated in **10** together with a **graph-first timing grid** placeholder—no per-variant README table yet for graph-first, so those cells are marked TODO until we freeze a comparable breakout.

9. Further research

- **Multi-GPU (2 vs. 4 GPUs per node).** Re-run the same validation and timing harness on Perlmutter jobs that allocate two and four A100s, keeping a simple 1D CSR vertex split and exchanging boundary frontiers with NCCL (or MPI). The goal is an apples-to-apples curve: how much of the single-GPU advantage survives once frontier traffic and load imbalance cross NVLink, and whether 4-GPU runs amortize setup enough to beat 2-GPU on identical budgets.
- **Warp–bitmap fused SpMV (not emphasized on the poster).** The repo implements `warpbitmap` for linear SpMV BFS, but we lead with the four standalone kernels here. Next step is a focused study: when does fusing cooperative row scans with a bitmap frontier beat the best non-fused pick, how does it interact with push–pull scheduling, and on which graphs does the extra 32× thread launch footprint dominate? That lines up with README tables once graph-first has a matching breakdown.
- **Harder inputs.** Fold in SNAP / GAP / road instances and compare against **cuGraph** once multi-GPU baselines exist.

10. Runtime (kernel ms)

Linear-algebraic SpMV (Perlmutter A100; matches `linear-algebraic/README.md`). Times in ms; **bold** = best per row.

	$\bar{d}$	dep	Bsln	Wrp	Bmp	P/P	W+B
Medium (10^3 vertices)							
`m_chain`	2	999	20.19	21.57	**19.02**	32.11	23.95
`m_dense`	100	2	5.30	7.02	0.58	0.65	**0.57**
`m_sparse`	6	6	2.47	**0.61**	0.63	0.79	0.69
Large (10^5 vertices)							
`l_dense`	100	4	4.76	14.00	4.44	4.67	**4.41**
`l_power`	40	3	2.81	2.56	2.46	4.38	**2.41**
`l_sparse`	10	7	11.34	1.79	18.15	1.79	**1.58**

Graph-first (direction-optimizing); per-kernel table TODO—same graph names, to be filled from the graph-first driver.

	$\bar{d}$	dep	top-down	bottom-up	hybrid
`m_chain`	2	999	TODO	TODO	TODO
`m_dense`	100	2	TODO	TODO	TODO
`m_sparse`	6	6	TODO	TODO	TODO
`l_dense`	100	4	TODO	TODO	TODO
`l_power`	40	3	TODO	TODO	TODO
`l_sparse`	10	7	TODO	TODO	TODO

11. References & Code

References

[1] S. Beamer, K. Asanović, D. Patterson. *Direction-Optimizing BFS.* SC '12.

[2] D. Merrill, M. Garland, A. Grimshaw. *Scalable GPU Graph Traversal.* PPoPP '12.

[3] J. Kepner et al. *Mathematical Foundations of the GraphBLAS.* HPEC '16.

[4] S. Beamer et al. *The GAP Benchmark Suite.* arXiv:1508.03619, 2015.

[5] NVIDIA RAPIDS. *cuGraph: GPU graph analytics.* `github.com/rapidsai/cugraph`.

Computations on NERSC Perlmutter *(NVIDIA A100, `sm_80`). Source, build scripts, validation harness:* `github.com/Harshaan-Chugh/parallel-bfs`.

LaTeX TikZposter